Pentium M Style Hybrid Branch Predictor

COSC6385 Computer Architecture — Branch Predictor Assignment

ChengAo Shen
Student ID: 2437132

October 2026

Code: `https://github.com/ChengAoShen/cosc6385-branch-predictor`

1 Summary

I implemented the direction predictor of the Pentium M (`pm_predictor` in `src/my_predictor.h`) on the CBP-2 infrastructure. It combines a bimodal table with a tagged, 4-way set-associative global predictor. A hit in the global table overrides the bimodal prediction.

Predictor	Storage	Average MPKI (20 traces)
Always taken (original `pm_predictor`)	0	86.312
Sample gshare (32K entries, 15-bit history)	8 KB	6.306
This design	**4.75 KB**	**5.495**

The final design has 12.9% fewer mispredictions than gshare with about 60% of its storage, and it is better than gshare on 18 of 20 traces.

Not implemented (allowed by the assignment): the loop predictor, target prediction, and the PIR hash. The global index uses `address XOR history` instead of the PIR.

2 Design

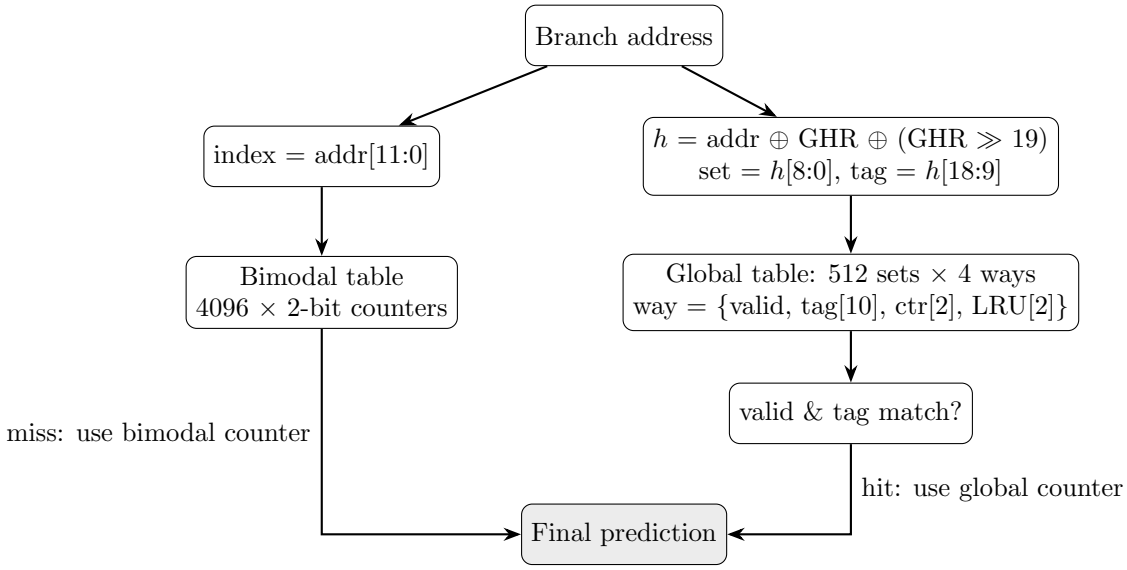


Figure 1: Structure of the predictor.

2.1 Tables and storage

Structure	Size	Bits
Bimodal table	4096 × 2-bit counters	8,192
Global table	512 sets × 4 ways × (1 valid + 10 tag + 2 counter + 2 LRU)	30,720
Global history register	21 bits	21
Total		**38,933 (≈ 4.75 KB)**

The table sizes (4K bimodal entries, 2K global entries, 4 ways) follow the Pentium M structure described by Milenković et al. [1].

2.2 Index and tag

- Bimodal index: `addr & 0xFFF` (low 12 bits of the branch address).
- Global hash: $h = \text{addr} \oplus \text{GHR} \oplus (\text{GHR} \gg 19)$; set = h[8:0] (512 sets), tag = h[18:9] (10 bits).
- Set and tag together use 19 bits. The GHR has 21 bits, so the GHR $\gg$ 19 term folds the two oldest history bits back into the hash. Without folding, history bits above bit 18 have no effect.

2.3 Prediction

1. If the branch is not conditional, predict taken (as in the original code).
2. Compute the bimodal index, the global set and the global tag, and save them in `pm_update` for the update step.
3. Search the 4 ways of the set for an entry with `valid = 1` and a matching tag.
4. Hit: direction = MSB of the global counter. Miss: direction = MSB of the bimodal counter.

2.4 Update (conditional branches only)

1. Bimodal counter: always updated (2-bit saturating, 0–3).
2. Global hit: update the counter of the hit way and make that way MRU.
3. Global miss: if the bimodal prediction was wrong, allocate the LRU way (valid = 1, write the tag, counter = `taken ? 2 : 1`, make it MRU). If bimodal was correct, do not allocate (Pentium M policy).
4. GHR $\leftarrow ((\text{GHR} \ll 1) \,|\, \text{taken}) \bmod 2^{21}$.

Non-conditional branches do not change counters or the GHR, because only conditional-branch direction misses are counted. LRU uses a 2-bit age per way (0 = most recently used); the victim is the way with age 3. Initially all bimodal counters are 1 (weakly not taken) and all global entries are invalid.

3 Results

MPKI = mispredictions per 1000 instructions (lower is better).

Trace	Always taken	gshare	This design
164.gzip	70.025	12.473	12.574
175.vpr	85.312	13.415	**12.900**
176.gcc	86.071	11.254	**8.279**
181.mcf	102.497	15.837	16.560
186.crafty	48.651	5.837	**5.190**
197.parser	78.421	10.008	**8.259**
201.compress	76.983	7.831	**7.366**
202.jess	92.343	1.562	**0.910**
205.raytrace	109.506	2.756	**1.861**
209.db	91.451	3.909	**3.354**
213.javac	109.644	2.267	**2.029**
222.mpegaudio	110.339	2.188	**2.013**
227.mtrt	112.022	2.657	**1.957**
228.jack	77.958	3.033	**1.808**
252.eon	24.226	1.807	**1.273**
253.perlbmk	66.833	2.554	**1.329**
254.gap	75.950	3.926	**2.538**
255.vortex	70.524	1.222	**0.706**
256.bzip2	178.257	0.094	**0.056**
300.twolf	59.221	21.489	**18.939**
Average	86.312	6.306	**5.495**

Table 1: Per-trace MPKI. Bold = better than gshare.

gzip and mcf are slightly worse than gshare. These traces have many data-dependent branches, and gshare's 32K counters suffer less from capacity pressure.

4 Parameter Study

The starting point is the configuration suggested in the homework guide: 4K bimodal, 512 sets $\times$ 4 ways, 6-bit tag, 15-bit history, no folding, allocation only on a bimodal miss (5.954). One

parameter is changed at a time unless noted.

Change	Avg. MPKI	Note
Baseline	5.954	
Allocate on every global miss	6.700	pollutes the table with easy branches
2 ways / 8 ways	6.466 / 5.767	
256 / 1024 / 2048 sets	6.348 / 5.689 / 5.527	2048 sets = 4× global storage
Tag 4 / 8 / 10 bits	6.786 / 5.803 / 5.766	short tags give false hits
Bimodal 1K / 16K	6.206 / 5.872	
History 8 / 12 bits	7.317 / 6.450	too little history
History 18 / 24 bits (no folding)	5.954 / 5.954	extra bits never reach the hash
Tag 8, history 18 / 21, folded	5.603 / 5.563	
Tag 10, history 15 / 18, folded	5.766 / 5.707	
Tag 10, history 21, folded (final)	**5.495**	
2048 sets × 8 ways, 16K bimodal	4.914	≈33 KB, 7× storage

Findings:

1. **Allocation policy matters most.** Allocating only when the bimodal predictor is wrong keeps easy branches out of the global table. Always allocating costs about +0.75 MPKI.
2. **Tag width controls aliasing.** A 4-bit tag causes false hits; 10 bits is enough.
3. **History helps only if it reaches the hash.** With set + tag = 15 bits, a longer history had no effect until the extra bits were folded in.
4. **Bigger tables help with diminishing returns.** I kept the Pentium M sizes instead of the 33 KB configuration.

5 How to Reproduce

```
make -C src
./run.sh traces          # prints per-trace MPKI and the average
```

`run.sh` is a bash version of the original `run` script (which needs `csh` and `dc`). Each parameter is a macro (`PM_BIM_BITS`, `PM_SET_BITS`, `PM_WAYS`, `PM_TAG_BITS`, `PM_HIST_LEN`, `PM_FOLD`, `PM_ALLOC`) and can be set with `-D` at compile time.

References

[1] M. Milenković, A. Milenković, and J. Kulick, "Demystifying Intel Branch Predictors," Workshop on Duplicating, Deconstructing, and Debunking (WDDD), 2002.

[2] The 2nd JILP Championship Branch Prediction Competition (CBP-2) simulation infrastructure and traces.

[3] S. McFarling, "Combining Branch Predictors," WRL Technical Note TN-36, 1993.

[4] COSC6385 lecture slides, Lecture 6 & 7: Branch Prediction.